



University of Piraeus

School of Information and Telecommunications Technologies

Department of Digital Systems

Level: Postgraduate Program of Study

Cyber Defense and Digital Forensics

Assignment 2

Supervising Professors: Christoforos Dantoyan and Evangelos Dragonas

Kalaitzidis Ioannis

ioannis_kalaitzidis@ssl-unipi.gr

Piraeus

10/05/2026

Exercise A – Volatility

- 1) In the memory copy you will find information about the current processes of the system (PIDs, PPIDs). Fill in the missing information in the table below:

Answer:

To solve the first query and analyze the memory dump through the Volatility 3 tool, the following preparation and installation process was followed in the Linux (Ubuntu) environment:

- The commands **sudo apt update** and **sudo apt install git python3 python3-pip python3-venv -y** were executed. The use of sudo ensured execution with administrator privileges, git was used to download the source code, while python3 (the language on which Volatility is based) was installed along with the pip package manager and the venv virtual environment builder.
- Then, to download the tool itself, the **git clone** command **<https://github.com/volatilityfoundation/volatility3.git>** was executed. Through it, a local copy of Volatility 3 was created from GitHub on our computer.
- In order to avoid any conflicts with the already installed libraries of the operating system, it was deemed necessary to create an isolated Python environment. Switched to the tool folder (cd volatility3) and executed the **python3 -m venv venv** commands to create the environment and **source venv/bin/activate** for its activation. Upon activation, the indicator (venv) confirmed the successful transition to the isolated environment.

```
kalaitzon@ubuntudesktop1:~$ git clone https://github.com/volatilityfoundation/volatility3.git
Cloning into 'volatility3'...
remote: Enumerating objects: 50517, done.
remote: Counting objects: 100% (596/596), done.
remote: Compressing objects: 100% (275/275), done.
remote: Total 50517 (delta 468), reused 322 (delta 321), pack-reused 49921 (from 2)
Receiving objects: 100% (50517/50517), 10.21 MiB | 6.51 MiB/s, done.
Resolving deltas: 100% (39168/39168), done.
kalaitzon@ubuntudesktop1:~$ cd volatility3
kalaitzon@ubuntudesktop1:~/volatility3$ python3 -m venv venv
kalaitzon@ubuntudesktop1:~/volatility3$ source venv/bin/activate
```

- After activating the virtual environment, the necessary libraries for the proper functioning of Volatility 3 were installed via the **pip install -e "[command** The **-e** (editable mode) option was used to install the tool in editable form, linking the environment directly to the local directory of the source code so that changes were reflected immediately without the need for reinstallation. At the same time, the **[full]** parameter ensured that the full suite of dependencies was downloaded, including all the optional libraries necessary to support the features and specialized plugins (plugins) of the tool.
- With the environment fully configured, we proceeded to analyze the memory file. To retrieve the list of current processes running on the system at the time of downloading the memory dump, we used the following **python3 command vol.py -f ~/Desktop/Investigation/windows.vmem windows.pslist**. **-f** was used to specify the exact path of the memory file (windows.vmem). The plugin windows.pslist (Process List) was used to read Windows memory structures (such as the EPROCESS list, which is the most important data structure within the Windows kernel. For each program that runs (e.g., chrome.exe or cmd.exe), the operating system creates a huge "box" in the RAM) and return the active processes (PIDs) and their parent processes (PPIDs).

By analyzing the results of the windows.pslist command, we identified the chrome.exe process which had a Process ID (PID) **1344** and Parent Process ID (PPID) **2188**. Looking then for which process corresponds to PID 2188, we found that it is the explorer.exe. Similarly, for Ipconfig.exe, we identified that he had a PID **2916** and PPID **1584**. Checking the list, PID 1584 corresponded to process cmd.exe.

kafatze@ubuntu:~/Desktop - ssh - /usr/bin/ssh										
PID	PPID	ImageFileName	Offset(V)	Threads	Handles	SessionId	Wow64	CreateTime	ExitTime	File output
0	0	System	0x00000000	0	0	0	False	2020-07-22 09:02:10.000000 UTC	N/A	Disabled
264	4	smss.exe	0x00000000	2	29	N/A	False	2020-07-22 09:02:10.000000 UTC	N/A	Disabled
352	348	csrss.exe	0x00000000	9	486	0	False	2020-07-22 09:02:21.000000 UTC	N/A	Disabled
404	348	wininitL.exe	0x00000000	3	74	0	False	2020-07-22 09:02:24.000000 UTC	N/A	Disabled
412	396	csrss.exe	0x00000000	11	197	1	False	2020-07-22 09:02:24.000000 UTC	N/A	Disabled
560	396	winlogon.exe	0x00000000	4	111	1	False	2020-07-22 09:02:24.000000 UTC	N/A	Disabled
584	404	services.exe	0x00000000	10	218	0	False	2020-07-22 09:02:24.000000 UTC	N/A	Disabled
512	404	lsass.exe	0x00000000	7	555	0	False	2020-07-22 09:02:24.000000 UTC	N/A	Disabled
520	404	lsass.exe	0x00000000	11	146	0	False	2020-07-22 09:02:24.000000 UTC	N/A	Disabled
624	504	svchost.exe	0x00000000	11	358	0	False	2020-07-22 09:02:24.000000 UTC	N/A	Disabled
600	504	svchost.exe	0x00000000	4	53	0	False	2020-07-22 09:02:24.000000 UTC	N/A	Disabled
720	504	svchost.exe	0x00000000	7	273	0	False	2020-07-22 09:02:25.000000 UTC	N/A	Disabled
820	504	svchost.exe	0x00000000	21	464	0	False	2020-07-22 09:02:25.000000 UTC	N/A	Disabled
880	504	svchost.exe	0x00000000	23	514	0	False	2020-07-22 09:02:27.000000 UTC	N/A	Disabled
900	504	svchost.exe	0x00000000	36	988	0	False	2020-07-22 09:02:27.000000 UTC	N/A	Disabled
980	820	audiodg.exe	0x00000000	7	129	0	False	2020-07-22 09:02:27.000000 UTC	N/A	Disabled
816	504	svchost.exe	0x00000000	12	518	0	False	2020-07-22 09:02:27.000000 UTC	N/A	Disabled
568	504	svchost.exe	0x00000000	21	380	0	False	2020-07-22 09:02:29.000000 UTC	N/A	Disabled
1156	504	spoolsv.exe	0x00000000	13	265	0	False	2020-07-22 09:02:30.000000 UTC	N/A	Disabled
1184	504	svchost.exe	0x00000000	21	325	0	False	2020-07-22 09:02:30.000000 UTC	N/A	Disabled
1364	504	VGAAuthService.exe	0x00000000	4	86	0	False	2020-07-22 09:02:31.000000 UTC	N/A	Disabled
1388	504	vmtoolsd.exe	0x00000000	10	269	0	False	2020-07-22 09:02:31.000000 UTC	N/A	Disabled
1836	624	winPrvSE.exe	0x00000000	10	197	0	False	2020-07-22 09:02:36.000000 UTC	N/A	Disabled
1888	504	dllhost.exe	0x00000000	16	195	0	False	2020-07-22 09:02:37.000000 UTC	N/A	Disabled
1900	504	cmd.exe	0x00000000	15	153	0	False	2020-07-22 09:02:38.000000 UTC	N/A	Disabled
1640	624	winPrvSE.exe	0x00000000	13	382	0	False	2020-07-22 09:02:56.000000 UTC	N/A	Disabled
204	900	taskeng.exe	0x00000000	5	75	0	False	2020-07-22 09:03:10.000000 UTC	N/A	Disabled
264	504	svchost.exe	0x00000000	10	140	0	False	2020-07-22 09:04:13.000000 UTC	N/A	Disabled
2460	504	sppsvc.exe	0x00000000	4	144	0	False	2020-07-22 09:04:34.000000 UTC	N/A	Disabled
2536	2500	GoogleCrashHan	0x00000000	4	97	0	True	2020-07-22 09:04:35.000000 UTC	N/A	Disabled
2544	2500	GoogleCrashHan	0x00000000	6	90	0	False	2020-07-22 09:04:35.000000 UTC	N/A	Disabled
2556	504	svchost.exe	0x00000000	14	332	0	False	2020-07-22 09:04:35.000000 UTC	N/A	Disabled
2688	504	SearchIndexer.exe	0x00000000	14	598	0	False	2020-07-22 09:04:35.000000 UTC	N/A	Disabled
3004	504	ipmapi.sys	0x00000000	5	113	0	False	2020-07-22 09:05:02.000000 UTC	N/A	Disabled
1624	504	taskhost.exe	0x00000000	10	156	1	False	2020-07-22 09:05:19.000000 UTC	N/A	Disabled
1540	880	dm.exe	0x00000000	4	69	1	False	2020-07-22 09:05:19.000000 UTC	N/A	Disabled
2188	596	explorer.exe	0x00000000	36	818	1	False	2020-07-22 09:05:19.000000 UTC	N/A	Disabled
2420	2188	vmtoolsd.exe	0x00000000	0	196	1	False	2020-07-22 09:05:20.000000 UTC	N/A	Disabled
1344	2188	chrome.exe	0x00000000	0	-	1	False	2020-07-22 09:06:37.000000 UTC	2020-07-22 09:07:03.000000 UTC	Disabled
3000	880	WUDFHost.exe	0x00000000	10	201	0	False	2020-07-22 09:07:29.000000 UTC	N/A	Disabled
1800	2688	SearchProtocol	0x00000000	7	227	1	False	2020-07-22 09:07:30.000000 UTC	N/A	Disabled
860	2688	SearchFilterHo	0x00000000	5	81	0	False	2020-07-22 09:07:30.000000 UTC	N/A	Disabled
1504	1300	cmd.exe	0x00000000	0	-	0	False	2020-07-22 09:07:57.000000 UTC	2020-07-22 09:07:57.000000 UTC	Disabled
3000	352	cmd.exe	0x00000000	0	-	0	False	2020-07-22 09:07:57.000000 UTC	2020-07-22 09:07:57.000000 UTC	Disabled
916	1504	ipconfig.exe	0x00000000	0	-	0	False	2020-07-22 09:07:57.000000 UTC	2020-07-22 09:07:57.000000 UTC	Disabled

Process ID (PID)	Parent Process ID (PPID)	Parent's Process ImageFileName	ImageFileName
1344	2188	explorer.exe	chrome.exe
2916	1584	cmd.exe	ipconfig.exe

2) What is the password of the user "Adam"?

Answer:

In Windows operating systems, user passwords are never stored in plain text for obvious security reasons. Instead, they are converted through cryptographic algorithms and stored in hash format (specifically as NTLM hash). Therefore, in order to recover the password of the user "Adam", the process was divided into two sub-stages, the extraction of the hash from memory and its subsequent decryption.

To recover the encrypted codes, Volatility 3's windows.hashdump plugin was leveraged. This plugin is designed to retrieve the hashes of the codes, extracting them directly from the Windows Registry and more specifically from the SYSTEM and SAM keys. In the active virtual environment, the following command was executed **python3 vol.py -f ~/Desktop/Investigation/windows.vmem windows.hashdump**.

Analyzing the result of the above command, the record corresponding to the user "Adam"

```
(vmenv) kalaitzon@kali:~/Desktop/Investigation$ python3 vol.py -f ~/Desktop/Investigation/windows.vmem windows.hashdump
Volatility 3 Framework 2.28.1
WARNING: volatility3.framework.layers.vmemware: No metadata file found alongside VMEM file. A VMSS or VMSSN file may be required to correctly process a VMEM file. These should be placed in the same directory with the same file name, e.g. windows.vmem and windows.vmemss.
/home/kalaitzon/volatility3/volatility3/framework/deprecation.py:28: FutureWarning: This API (volatility3.plugins.windows.registry.hashdump.Hashdump.run) will be removed in the first release after 2026-09-25. This plugin has been renamed; please call volatility3.plugins.windows.registry.hashdump.Hashdump rather than volatility3.plugins.windows.hashdump.Hashdump.
  warnings.warn(

User      rid      lmhash      nthash
/home/kalaitzon/volatility3/volatility3/framework/deprecation.py:185: FutureWarning: This plugin (volatility3.plugins.windows.hashdump.Hashdump) has been renamed and will be removed in the first release after 2026-09-25. Please ensure all method calls to this plugin are replaced with calls to volatility3.plugins.windows.registry.hashdump.Hashdump
  warnings.warn(
Administrator  500      aad3b435b51404eeaad3b435b51404ee  31d6cf8d16ae931b73c59d7e0c889c0
Guest  501      aad3b435b51404eeaad3b435b51404ee  31d6cf8d16ae931b73c59d7e0c889c0
Adam  1000     aad3b435b51404eeaad3b435b51404ee  7773c08920232397cae081704964b786
(vmenv) kalaitzon@kali:~/Desktop/Investigation$
```

was identified. The data of the record is analyzed as follows:

- **rid: 1000:** This is the unique user ID in the system.
- **lmhash: aad3b435b51404eeaad3b435b51404ee:** Contains a default, empty value that indicates that the system is not using the older (and vulnerable) method of storing LM codes (LAN Manager).
- **nthash: 7773c08920232397cae081704964b786:** This is the real and useful imprint of the code.

Therefore, the NTLM Hash of the user 'Adam' was identified as **7773c08920232397cae081704964b786**.

To find the real password in plain text format, the extracted NTLM hash was imported into the online decryption platform «[CrackStation](#)». This platform compares each hash with huge databases of budgeted hashes to find the original word.



As confirmed by the platform's result, the hash was successfully identified. **The password of user "Adam" is qwerty123.**

- 3) How many times has "Adam" run the "Windows Remote Desktop" application?

Answer:

To find the number of executions of the "Windows Remote Desktop" application, the information of the Windows Application Execution subsystem was used. On Windows systems, the Registry key responsible for recording the applications running, as well as the number of their runs, is called UserAssist. To read this data directly from the memory copy, Volatility 3's specialized plugin windows.registry.userassist was used.

Because the UserAssist subsystem often records execution via the shortcut (shortcut .lnk) selected by the user from the Start menu, and not necessarily by the name of the main executable (in this case mstsc.exe), direct search/filter by the executable (e.g. using grep) may not return results. For this reason, it was preferred to export (redirect) all the UserAssist data to a local text file for further thorough analysis. The following command was executed in the terminal:

```
python3 vol.py -f ~/Desktop/Investigation/windows.vmem windows.registry.userassist > userassist.txt
```

Specifically, the part of the command "python3 vol.py -f ~/Desktop/Investigation/windows.vmem" initiates the Volatility framework, specifying through the parameter "-f" the exact path of the memory copy being examined. The specialized plugin "windows.registry.userassist" is then called to retrieve the application execution data from the Registry structures. Finally, the parameter "> userassist.txt" instructed the operating system not to print the huge volume results on the terminal screen, but to create a new file named «userassist.txt" and store all the output there for easier analysis. By opening the exported userassist.txt file with a copywriter, a search was performed for the term "Remote Desktop". The search successfully identified two related records belonging to the profile of the user "Adam", one for the classic shortcut (Microsoft\Windows\Start Menu\Programs\Accessories\Remote Desktop Connection.lnk) and one for the system application (Microsoft.Windows.RemoteDesktop).

Mapping row data to table headers (Name | ID | Count | Focus Count | Time Focused), the

```
* 0xf8a08756d010 \??\C:\Users\Adam\ntuser.dat ntuser.dat\Software\Microsoft\Windows\CurrentVersion\Explorer\UserAssist\{CEBFF5CD-ACE2-8F4F-9178-9926F41748FA}\Count 2020-07-22
09:07:49.000000 UTC Value Microsoft.Windows.RemoteDesktop N/A 8 0:03:00.500000 2020-07-20 13:21:19.000000 UTC
00 00 00 00 00 00 00 00 00 00 00 00 00 20 bf 02 00 .....
00 00 00 bf 00 00 00 bf 00 00 00 bf 00 00 00 bf .....
00 00 00 bf 00 00 00 bf 00 00 00 bf 00 00 00 bf .....
00 00 00 bf 00 00 00 bf ff ff ff ff 02 03 00 07 .....
00 5a 55 01 00 00 00 00 .....

0xf8a08756d010 \??\C:\Users\Adam\ntuser.dat ntuser.dat\Software\Microsoft\Windows\CurrentVersion\Explorer\UserAssist\{F3F57C40-293E-41F0-80AB-6A30CFE1309F}\Count 2020-07-22
09:06:37.000000 UTC Value %ALLUSERSPROFILE%\Microsoft\Windows\Start Menu\Programs\Accessories\Remote Desktop Connection.lnk N/A 8 0:00:00.500000 2020-07-20
13:21:19.000000 UTC
00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 .....
00 00 00 bf 00 00 00 bf 00 00 00 bf 00 00 00 bf .....
00 00 00 bf 00 00 00 bf 00 00 00 bf 00 00 00 bf .....
00 00 00 bf 00 00 00 bf ff ff ff ff 02 03 00 07 .....
00 5a 55 01 00 00 00 00 .....
```

following was found:

ID: N/A

Count: 8

Focus Count: 9

Therefore, based on the UserAssist data retrieved from memory, the integer that makes up the answer is 8. **"Adam" ran the "Windows Remote Desktop" app 8 times.**

4) When did the user "Adam" open the file "1.jpg"?

Answer:

To determine the exact time at which the user "Adam" opened the file «1.jpg», the research focused on the analysis of the file access history (File/Folder Opening). In Windows operating systems, every time a user opens a file, this action is recorded in the Registry, and in particular in the RecentDocs key, in order to feed the "Recent Documents" function.

To read and extract this specific key directly from the RAM copy, Volatility 3's "windows.registry.printkey" plugin was used. The search was specifically targeted at the subcategory of recording files ending in image .jpg. The following command was executed in the terminal:

```
python3 vol.py -f ~/Desktop/Investigation/windows.vmem windows.registry.printkey --key "Software\Microsoft\Windows\CurrentVersion\Explorer\RecentDocs\.jpg".
```

The above command instructed Volatility to locate the Registry hives in memory and print the contents of the specified path.

```

kali@kali:~/Desktop$ python3 vol.py -f /Desktop/investigation/windows.vmem windows.registry.printkey --key "Software\Microsoft\Windows\CurrentVersion\Explorer\RecentDocs\"
Volatility 3 Framework 2.20.1
WARNING: volatility.layers.vmware: No metadata file found alongside VMEM file. A VMSS or VMSN file may be required to correctly process a VMEM file. These should be placed in the same directory with the same file name, e.g., windows.vmem and windows.vmss..
Progress: 100.00 PDB scanning finished.
Last Write Time Hive Offset Type Key Name Data Volatile
--
0xfba0000fb010 Key [HKEY_CURRENT_CONFIG\Software\Microsoft\Windows\CurrentVersion\Explorer\RecentDocs].jpg
0xfba0000d24010 Key [HKEY_LOCAL_MACHINE\SYSTEM\Software\Microsoft\Windows\CurrentVersion\Explorer\RecentDocs].jpg
0xfba0000536010 Key [HKEY_LOCAL_MACHINE\SOFTWARE\Software\Microsoft\Windows\CurrentVersion\Explorer\RecentDocs].jpg
0xfba00078a010 Key [Device\HarddiskVolume1\Boot\BCD\Software\Microsoft\Windows\CurrentVersion\Explorer\RecentDocs].jpg
0xfba0013c3010 Key [SystemRoot\System32\Config\SOFTWARE\Software\Microsoft\Windows\CurrentVersion\Explorer\RecentDocs].jpg
0xfba00164d010 Key [SystemRoot\System32\Config\DEFAULT\Software\Microsoft\Windows\CurrentVersion\Explorer\RecentDocs].jpg
0xfba0019f6010 Key [SystemRoot\System32\Config\SECURITY\Software\Microsoft\Windows\CurrentVersion\Explorer\RecentDocs].jpg
0xfba001bf8010 Key [SystemRoot\System32\Config\SAM\Software\Microsoft\Windows\CurrentVersion\Explorer\RecentDocs].jpg
0xfba001993010 Key \??C:\Windows\ServiceProfiles\LocalService\NTUSER.DAT\Software\Microsoft\Windows\CurrentVersion\Explorer\RecentDocs].jpg
0xfba001a23010 Key \??C:\Windows\ServiceProfiles\NetworkService\NTUSER.DAT\Software\Microsoft\Windows\CurrentVersion\Explorer\RecentDocs].jpg
2020-07-21 18:38:33.000000 UTC 0xfba00256d010 REG_BINARY \??C:\Users\Admin\ntuser.dat\Software\Microsoft\Windows\CurrentVersion\Explorer\RecentDocs].jpg MMUListe
00 00 00 ff ff ff ff ff ff ff ff ff ff ff ff ff ff ff ff ff ff ff ff ff ff ff ff ff ff ff ff ff ff ff ff ff ff ff ff
7020-07-21 18:38:33.000000 UTC 0xfba00256d010 REG_BINARY \??C:\Users\Admin\ntuser.dat\Software\Microsoft\Windows\CurrentVersion\Explorer\RecentDocs].jpg #
31 00 2e 00 64 00 70 00 67 00 00 00 4c 00 32 00 1.....1...
00 00 00 00 00 00 00 00 31 2e 4c 60 6b 00 .....1.lnk.
30 00 05 00 04 00 ef be 00 00 00 00 00 00 00 00 00 B.....
24 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 .....
00 00 00 00 00 00 00 00 00 00 31 00 2e 00 6c 00 .....1..l.
6e 00 5b 00 00 00 14 00 00 00 n.k..... False
0xfba002571010 Key \??C:\Users\Admin\AppData\Local\Microsoft\Windows\UserClass.dat\Software\Microsoft\Windows\CurrentVersion\Explorer\RecentDocs].jpg
(vvvv) kali@kali:~/Desktop$ python3 $

```

By analyzing the output of the command, the record associated with the user's profile "Adam" was successfully identified, as indicated by the path \\?\C:\Users\Adam\ntuser.dat\Software\Microsoft\Windows\CurrentVersion\Explorer\RecentDocs\1.jpg. In the record data block, the traces of the file name (1.jpg) and the corresponding shortcut (1.lnk) automatically generated by the system are clearly distinguishable. Along the same lines, Volatility extracted the exact timestamp of the action based on the time it was recorded in the Registry. Therefore, it is digitally proven that **file «1.jpg» was opened by "Adam" on 2020-07-21 18:38:33 UTC.**

5) When did the user search the internet for the phrase "How to hack everything"?

Answer:

In order to solve the fifth question, it was necessary to locate and dump the history database ("History" file) of the Google Chrome browser from the memory copy. This process was implemented in individual stages.

The first step involved finding the exact memory address (offset) in which the file was loaded. For this purpose, Volatility 3's "windows.filescan" plugin was used, which detects traces and file structures (FILE_OBJECTs) in memory. In order to limit the huge volume of results and to isolate only the records related to the history of the user "Adam", the output of the command was filtered in combination with the use of the grep command. Specifically, the following command was executed in the terminal: **python3 vol.py -f ~/Desktop/Investigation/windows.vmem windows.filescan | grep -i "Adam.*History"**. Through this targeted search, the record corresponding to the desired path of the file (\Users\Adam\AppData\Local\Google\Chrome\User Data\Default\History) was successfully located. To the left of the generated result bar, the memory address (offset) of the file was revealed, which was recorded as a necessary element for the next stage of the extraction.

From the execution of the above command in the terminal, three different results (routes) emerged. Of these, the third record was isolated and used exclusively, as its path was completely

```
(venv) kalattz@ubuntudesktop:~$ python3 vol.py -f ~/Desktop/Investigation/windows.vmem windows.filescan | grep -i "Adam.*History"
WARNING: volatility3.framework.layers.vmem: No metadata file found alongside VMEM file. A VMSS or VMEM file may be required to correctly process a VMEM file. These should be placed in the same directory with the same file name, e.g. windows.vmem and windows.vmemss.
0x3fb195d0 100,0\Users\Adam\AppData\Local\Google\Chrome\User Data\Default\History Provider Cache
0x3fd4a42e \Users\Adam\AppData\Local\Google\Chrome\User Data\Default\Media History
0x3fe3ae90 \Users\Adam\AppData\Local\Google\Chrome\User Data\Default\History
(venv) kalattz@ubuntudesktop:~$
```

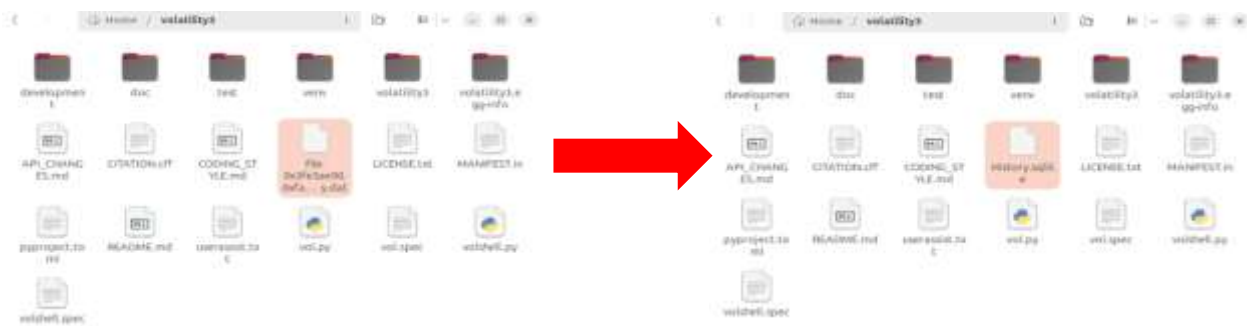
identical to the requested History file (unlike the other records that end up in Media History and Provider Cache). As shown to the left of the selected result bar, the exact memory address of the file was identified as 0x3fe3ae90. Having identified the specific address, the process proceeded to the next stage, which involved extracting the file from the memory copy and storing it locally in the system for further examination.

To extract the localized file from the memory copy, the plugin "windows.dumpfiles" was selected. The initial extraction attempt was made by considering the address as fictitious, executing

```
(venv) kalattz@ubuntudesktop:~$ python3 vol.py -f ~/Desktop/Investigation/windows.vmem windows.dumpfiles --physaddr 0x3fe3ae90
Volatility 3 Framework 2.28.1
WARNING: volatility3.framework.layers.vmem: No metadata file found alongside VMEM file. A VMSS or VMEM file may be required to correctly process a VMEM file. These should be placed in the same directory with the same file name, e.g. windows.vmem and windows.vmemss.
Progress: 100.00 PDB scanning finished
Cache FileObject FileName Result
DataSectionObject 0x3fe3ae90 History file.0x3fe3ae90.0xfa80cdc26b0.DataSectionObject.History.dat
(venv) kalattz@ubuntudesktop:~$
```

the following command: `python3 vol.py -f ~/Desktop/Investigation/windows.vmem windows.dumpfiles --virtaddr 0x3fe3ae90`. However, the command did not yield the expected results (blank output table). This error is due to the fact that the 0x3fe3ae90 address, which was previously retrieved by the filescan plug-in, is a physical address and not a virtual address. Recognizing this difference in memory architecture, the methodology was readjusted and the command was modified using the "--physaddr" parameter: `python3 vol.py -f ~/Desktop/Investigation/windows.vmem windows.dumpfiles --physaddr 0x3fe3ae90`. The execution of the corrected command was a complete success, exporting the database file to the local directory under the name `file.0x3fe3ae90.0xfa800cdc26b0.DataSectionObject.History.dat`.

In order to be able to analyze the contents of the extracted file, it was deemed necessary to rename it to "History.sqlite" so that it would be correctly recognized by the operating system as a database of SQLite format.



The file was then imported for further processing in the specialized DB Browser for SQLite application. Navigating to the "Browse Data" tab of the graphical interface, the "keyword_search_terms" table was analyzed. In this table, the record related to the search term 'How to hack everything' was successfully located. Examining the recording data, it was found that the timestamps are recorded by the Google Chrome browser in the WebKit timestamp formatting, which requires further conversion.

id	url	title	visit_count	repeat_count	last_visit_time	favicon
4	https://www.microsoft.com/en-us/	Microsoft - Official Home Page	2	0	13239826440787747	
5	https://www.google.com/	Google	2	0	13239826440787747	
6	https://www.facebook.com/	Facebook	2	0	13239826440787747	
7	https://www.instagram.com/	Instagram	2	0	13239826440787747	
8	https://www.linkedin.com/	LinkedIn	2	0	13239826440787747	
9	https://www.tumblr.com/	tumblr	2	0	13239826440787747	
10	https://www.youtube.com/	YouTube	2	0	13239826440787747	
11	https://www.spotify.com/	Spotify	2	0	13239826440787747	
12	https://www.netflix.com/	Netflix	2	0	13239826440787747	
13	https://www.amazon.com/	Amazon	2	0	13239826440787747	
14	https://www.ebay.com/	eBay	2	0	13239826440787747	
15	https://www.walmart.com/	Walmart	2	0	13239826440787747	
16	https://www.target.com/	Target	2	0	13239826440787747	
17	https://www.hugoboss.com/	HUGO BOSS	2	0	13239826440787747	
18	https://www.godaddy.com/	GoDaddy	2	0	13239826440787747	
19	https://www.10web.com/	10Web	2	0	13239826440787747	
20	https://www.godaddy.com/	GoDaddy	2	0	13239826440787747	
21	https://www.godaddy.com/	GoDaddy	2	0	13239826440787747	
22	https://www.godaddy.com/	GoDaddy	2	0	13239826440787747	
23	https://www.godaddy.com/	GoDaddy	2	0	13239826440787747	
24	https://www.godaddy.com/	GoDaddy	2	0	13239826440787747	
25	https://www.godaddy.com/	GoDaddy	2	0	13239826440787747	
26	https://www.godaddy.com/	GoDaddy	2	0	13239826440787747	
27	https://www.godaddy.com/	GoDaddy	2	0	13239826440787747	
28	https://www.godaddy.com/	GoDaddy	2	0	13239826440787747	
29	https://www.godaddy.com/	GoDaddy	2	0	13239826440787747	
30	https://www.godaddy.com/	GoDaddy	2	0	13239826440787747	

The output value (13239826440787747) is a WebKit timestamp. This formatting is natively used by Chromium-based browsers (such as Google Chrome) and records time in microseconds that have elapsed since January 1, 1601 (00:00:00 UTC). To convert this 17-digit value into a human-readable format (based on the UTC time zone required), a specialized online conversion tool (Chrome/WebKit Timestamp Converter) was used. The conversion revealed that the time corresponds to **Tuesday, July 21, 2020 at 5:34:00 PM (GMT/UTC).**

Convert Chrome/WebKit timestamps to human-readable date

The **WebKit Precision Time** or **High-Resolution Time** timestamp format is used by Google Chrome (Chromium) in its `base::Time` class.

It is a 64-bit signed integer (int64_t) representing **microseconds since January 1, 1601 00:00 UTC**. One microsecond is one-millionth of a second.

Google Chrome and other Chromium browsers like Microsoft Edge, Brave and Opera use this timestamp format internally for various purposes, such as tracking history, cookies, cache, and performance metrics.

You can find these timestamps in Chrome's `SQLite` databases, such as `History`, `Cookies`, and `Cache`. If you open the `History` file in a database viewer, the `visit_time`, `last_visit_time` fields are WebKit timestamps. In the `Cookies` database, the `expires_utc` and `creation_utc` fields are WebKit timestamps.

The current WebKit timestamp is **13422586398000000**

Enter a 17-digit Chrome/WebKit timestamp:

13239826440787747 **Chrome/WebKit timestamp to readable date**

GMT Tuesday, July 21, 2020 at 5:34:00 PM
UTC Tuesday, July 21, 2020 at 5:34:00 PM GMT+03:00
 Epoch/Unix time: 1595352840 (in seconds)

Therefore, the final answer for question 5 is **2020-07-21 17:34:00 UTC.**

Epilogue

Taking a quick look back at what we've accomplished with Volatility 3, we've now put together a complete data folder for the system and the 'Adam' user. First, through process analysis, we mapped the parent-child relationship for critical processes such as chrome.exe and ipconfig.exe. Proceeding to crack the code, we successfully extracted the NTLM Hash from memory and revealed that Adam's code was 'qwerty123.' In addition, by studying the execution history through the Registry (UserAssist), we proved that the user had opened the Remote Desktop application 8 times, while at the same time, analyzing the access to files, we identified the exact moment (2020-07-21 18:38:33 UTC) when the 1.jpg file was opened. Finally, in terms of browsing history, we dumped the entire Google Chrome database from RAM, bringing to light the suspicious search "How to hack everything" on 2020-07-21 17:34:00 UTC.

Exercise B – EvtxECmd

Before answering the individual questions, it was deemed necessary to prepare all the logs (Windows Event Logs Dataset) in order to make them readable and analysable. The process was divided into two main stages, converting the raw .evtx files and importing them into an analysis environment. To massively analyze and extract the data from Windows logs, the specialized command-line tool EvtxECmd was leveraged. In the Windows terminal (Command Prompt), the following command was executed **EvtxECmd.exe -d Logs --csv . --csvf ParsedLogs.csv**. The structure and function of this command are broken down as follows:

- **-d Logs**: This parameter (-d from the directory) indicated to the tool the directory ("Logs") in which the .evtx files to be examined were stored.
- **--csv .**: Instructed to export the processed data in comma-separated file format (CSV). The dot (.) specified that the exported file will be saved to the current working directory.
- **--csvf ParsedLogs.csv**: Specified the exact and final name of the generated file as ParsedLogs.csv.

After the command execution was complete, the tool successfully generated the aggregate file ParsedLogs.csv to the work folder.

```
C:\Windows\System32\cmd.exe
Microsoft Windows [Version 10.0.26298.8328]
(c) Microsoft Corporation. All rights reserved.

C:\Users\Nalal\Desktop\ExerciseB>EvtxECmd.exe -d Logs -csv - -csvf ParsedLogs.csv
EvtxECmd version 2026.0.0

Author: Eric Zimmerman (saericzimmerman@gmail.com)
https://github.com/EricZimmerman/evtxecmd

Command line: -d Logs -csv - -csvf ParsedLogs.csv

Warning: Administrator privileges not found!

CSV output will be saved to .\ParsedLogs.csv

Error loading map file C:\Users\Nalal\Desktop\ExerciseB\Maps\Microsoft-Windows-Storage-ClassPnP-Operational_Microsoft-Windows-StorDiag_597.map: An item with the same key has already been added.
System.ArgumentException: An item with the same key has already been added.
   at System.ThrowHelper.ThrowArgumentException(ExceptionResource resource)
   at System.Collections.Generic.Dictionary`2.Insert(TKey key, TValue value, Boolean add)
   at evtxec.EventLog.LoadMaps(String mapPath)
Error loading map file C:\Users\Nalal\Desktop\ExerciseB\Maps\Microsoft-Windows-VHDMF-Operational_Microsoft-Windows-VHDMF_1_Legacy.map: An item with the same key has already been added.
System.ArgumentException: An item with the same key has already been added.
   at System.ThrowHelper.ThrowArgumentException(ExceptionResource resource)
   at System.Collections.Generic.Dictionary`2.Insert(TKey key, TValue value, Boolean add)
   at evtxec.EventLog.LoadMaps(String mapPath)
Error loading map file C:\Users\Nalal\Desktop\ExerciseB\Maps\Microsoft-Windows-VHDMF-Operational_Microsoft-Windows-VHDMF_2_Legacy.map: An item with the same key has already been added.
System.ArgumentException: An item with the same key has already been added.
   at System.ThrowHelper.ThrowArgumentException(ExceptionResource resource)
   at System.Collections.Generic.Dictionary`2.Insert(TKey key, TValue value, Boolean add)
   at evtxec.EventLog.LoadMaps(String mapPath)
Maps loaded: 488
Looking for event log files in Logs

Processing C:\Users\Nalal\Desktop\ExerciseB\Logs\Application.evtx...
Chunk count: 1, Iterating records...
Record #: 894 (timestamp: 2024-12-27 00:06:35.5553605): Warning! Time just went backwards! Last seen time before change: 2024-12-27 00:06:36.5724283
Record #: 895 (timestamp: 2024-04-03 00:14:23.7715584): Warning! Time just went backwards! Last seen time before change: 2024-12-27 00:09:43.7698389
Record #: 1002 (timestamp: 2024-01-03 01:18:07.0850285): Warning! Time just went backwards! Last seen time before change: 2024-04-03 00:14:54.031740
#
```

```
C:\Windows\System32\cmd.exe

7824      1
7848      2
10016     2
10062     1
10077     1
10083     1
10095     1
10097     2
10103     1
10104     2
10105     2
10106     2
10146     1
10147     2
10157     2

Processing C:\Users\Nalal\Desktop\ExerciseB\Logs\Windows PowerShell.evtx...
Chunk count: 1, Iterating records...

Event log details:
Flags: None
Chunk count: 1
Stored/Calculated CRC: 198388EC/198388EC
Earliest timestamp: 2024-10-29 12:13:28.0543565
Latest timestamp: 2024-12-24 12:58:47.1505326
Total event log records found: 16

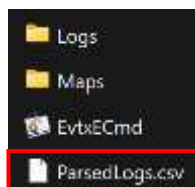
Records included: 16 Errors: 0 Events dropped: 0

Metrics (including dropped events)
Event ID      Count
-----
1000          2
1003          2
1008         12

Processed 116 files in 13.8804 seconds

Files with errors
C:\Users\Nalal\Desktop\ExerciseB\Logs\Microsoft-Windows-CertificateServicesClient-Lifecycle-System%40Operational.evtx error count: 1

C:\Users\Nalal\Desktop\ExerciseB>
```



Given the huge volume of records that resulted, the Timeline Explorer software was used to efficiently review, filter and search the digital evidence. The generated «ParsedLogs.csv" file was imported into the application environment, which undertook the categorization of the data into structured columns (such as Event ID, Time Created, Provider, Payload, etc.), making it ready for the digital forensic analysis that follows in the following questions.

Line	Tag	Record Number	Event Record Id	Time Created	Event Id	Level	Provider	Channel	Process
1		957	957	2024-12-24 23...	6000	Info	Microsoft-Windows-Winlogon	Application	
2		958	958	2024-12-24 23...	6000	Info	Microsoft-Windows-Winlogon	Application	
3		959	959	2024-12-24 23...	1532	Info	Microsoft-Windows-User Profiles Service	Application	
4		968	968	2024-12-27 00...	1531	Info	Microsoft-Windows-User Profiles Service	Application	
5		961	961	2024-12-27 00...	100	Info	Bonjour Service	Application	
6		962	962	2024-12-27 00...	100	Info	Bonjour Service	Application	
7		963	963	2024-12-27 00...	100	Info	Bonjour Service	Application	
8		964	964	2024-12-27 00...	5615	Info	Microsoft-Windows-NHI	Application	
9		965	965	2024-12-27 00...	6003	Info	Microsoft-Windows-Winlogon	Application	
10		966	966	2024-12-27 00...	6000	Info	Microsoft-Windows-Winlogon	Application	
11		967	967	2024-12-27 00...	900	Info	Microsoft-Windows-Security-SPP	Application	
12		968	968	2024-12-27 00...	16394	Info	Microsoft-Windows-Security-SPP	Application	
13		969	969	2024-12-27 00...	1866	Info	Microsoft-Windows-Security-SPP	Application	
14		970	970	2024-12-27 00...	1803	Info	Microsoft-Windows-Security-SPP	Application	
15		971	971	2024-12-27 00...	902	LogAlways	Microsoft-Windows-Security-SPP	Application	
16		972	972	2024-12-27 00...	5617	Info	Microsoft-Windows-NHI	Application	
17		973	973	2024-12-27 00...	1803	Info	Microsoft-Windows-Security-SPP	Application	
18		974	974	2024-12-27 00...	1803	Info	Microsoft-Windows-Security-SPP	Application	
19		975	975	2024-12-27 00...	1803	Info	Microsoft-Windows-Security-SPP	Application	
20		976	976	2024-12-27 00...	1803	Info	Microsoft-Windows-Security-SPP	Application	
21		977	977	2024-12-27 00...	8198	Error	Microsoft-Windows-Security-SPP	Application	
22		978	978	2024-12-27 00...	1803	Info	Microsoft-Windows-Security-SPP	Application	
23		979	979	2024-12-27 00...	8198	Error	Microsoft-Windows-Security-SPP	Application	
24		980	980	2024-12-27 00...	1803	Info	Microsoft-Windows-Security-SPP	Application	
25		981	981	2024-12-27 00...	8198	Error	Microsoft-Windows-Security-SPP	Application	
26		982	982	2024-12-27 00...	102	Info	ESENT	Application	

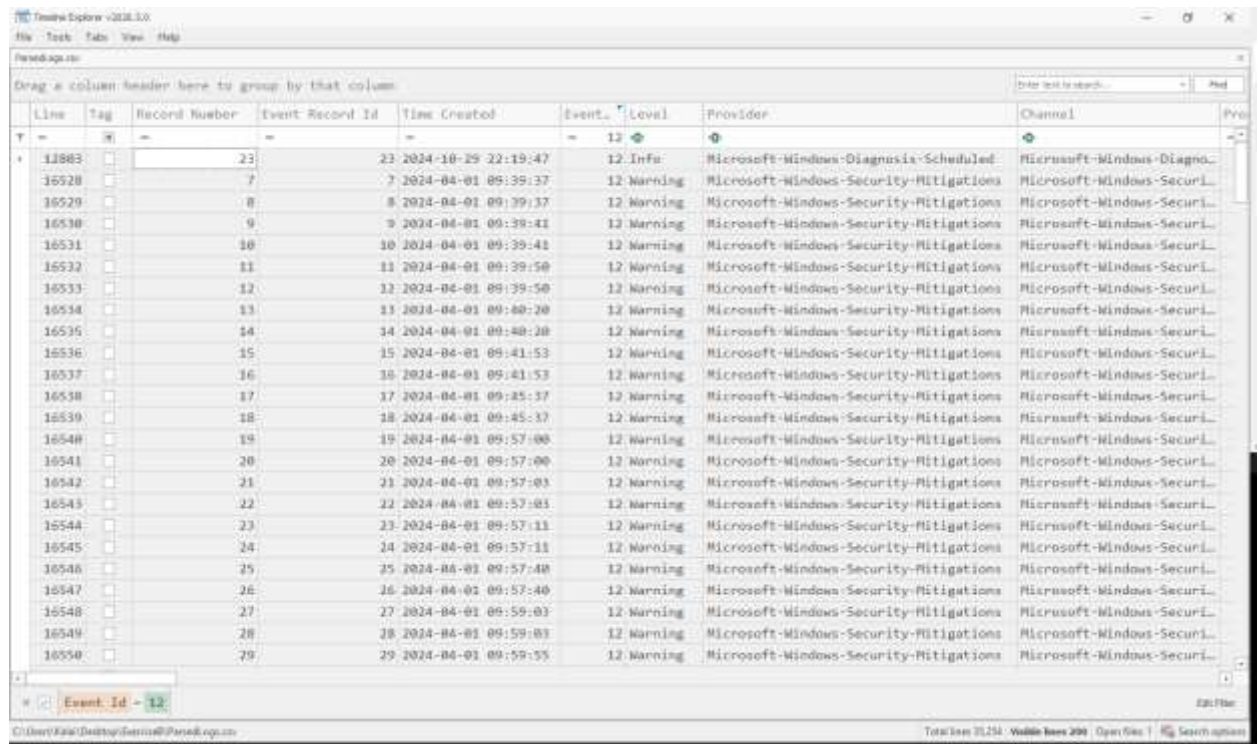
- 1) What is the recorded start time (OS start time) and shutdown time (OS shutdown time) of the system?

Answer:

In order to find the recorded OS start time and OS shutdown time of the system, the analysis in the Timeline Explorer environment focused on searching for specific event IDs (Event IDs). According to the official Windows documentation, the boot of the operating system is recorded with Event ID 12, while the shutdown is recorded with Event ID 13.

During the initial search solely based on Event ID 12, the tool returned a plethora of results. Analyzing the Provider column, it was found that these records came from unrelated

subsystems (such as Microsoft-Windows-Security-Mitigations). This is due to the fact that in Windows operating systems, the numbering of Event IDs is not completely unique throughout the system, but their semantics depend directly on the Provider that generates the respective event.



Line	Tag	Record Number	Event Record Id	Time Created	Event ID	Level	Provider	Channel	Pro
13883		23	23	2024-10-25 22:19:47	12	Info	Microsoft-Windows-Diagnosis-Scheduled	Microsoft-Windows-Diagno...	
16528		7	7	2024-04-01 09:39:37	12	Warning	Microsoft-Windows-Security-Mitigations	Microsoft-Windows-Securi...	
16529		8	8	2024-04-01 09:39:37	12	Warning	Microsoft-Windows-Security-Mitigations	Microsoft-Windows-Securi...	
16530		9	9	2024-04-01 09:39:41	12	Warning	Microsoft-Windows-Security-Mitigations	Microsoft-Windows-Securi...	
16531		10	10	2024-04-01 09:39:41	12	Warning	Microsoft-Windows-Security-Mitigations	Microsoft-Windows-Securi...	
16532		11	11	2024-04-01 09:39:50	12	Warning	Microsoft-Windows-Security-Mitigations	Microsoft-Windows-Securi...	
16533		12	12	2024-04-01 09:39:50	12	Warning	Microsoft-Windows-Security-Mitigations	Microsoft-Windows-Securi...	
16534		13	13	2024-04-01 09:40:20	12	Warning	Microsoft-Windows-Security-Mitigations	Microsoft-Windows-Securi...	
16535		14	14	2024-04-01 09:40:30	12	Warning	Microsoft-Windows-Security-Mitigations	Microsoft-Windows-Securi...	
16536		15	15	2024-04-01 09:41:53	12	Warning	Microsoft-Windows-Security-Mitigations	Microsoft-Windows-Securi...	
16537		16	16	2024-04-01 09:41:53	12	Warning	Microsoft-Windows-Security-Mitigations	Microsoft-Windows-Securi...	
16538		17	17	2024-04-01 09:45:37	12	Warning	Microsoft-Windows-Security-Mitigations	Microsoft-Windows-Securi...	
16539		18	18	2024-04-01 09:45:37	12	Warning	Microsoft-Windows-Security-Mitigations	Microsoft-Windows-Securi...	
16540		19	19	2024-04-01 09:57:00	12	Warning	Microsoft-Windows-Security-Mitigations	Microsoft-Windows-Securi...	
16541		20	20	2024-04-01 09:57:00	12	Warning	Microsoft-Windows-Security-Mitigations	Microsoft-Windows-Securi...	
16542		21	21	2024-04-01 09:57:03	12	Warning	Microsoft-Windows-Security-Mitigations	Microsoft-Windows-Securi...	
16543		22	22	2024-04-01 09:57:03	12	Warning	Microsoft-Windows-Security-Mitigations	Microsoft-Windows-Securi...	
16544		23	23	2024-04-01 09:57:11	12	Warning	Microsoft-Windows-Security-Mitigations	Microsoft-Windows-Securi...	
16545		24	24	2024-04-01 09:57:11	12	Warning	Microsoft-Windows-Security-Mitigations	Microsoft-Windows-Securi...	
16546		25	25	2024-04-01 09:57:40	12	Warning	Microsoft-Windows-Security-Mitigations	Microsoft-Windows-Securi...	
16547		26	26	2024-04-01 09:57:40	12	Warning	Microsoft-Windows-Security-Mitigations	Microsoft-Windows-Securi...	
16548		27	27	2024-04-01 09:59:03	12	Warning	Microsoft-Windows-Security-Mitigations	Microsoft-Windows-Securi...	
16549		28	28	2024-04-01 09:59:03	12	Warning	Microsoft-Windows-Security-Mitigations	Microsoft-Windows-Securi...	
16550		29	29	2024-04-01 09:59:55	12	Warning	Microsoft-Windows-Security-Mitigations	Microsoft-Windows-Securi...	

To accurately isolate the actual start and stop events, combined filtering was applied. The search was limited to Event IDs 12 and 13 which come exclusively from the provider **Microsoft-Windows-Kernel-General**, which represents the kernel of the operating system.

Timeline Explorer v2023.10

File Task Filter View Help

Timeline Explorer

Drag a column header here to group by that column

Enter text to search

Find

Line	Tag	Record Number	Event Record Id	Time Created	Event	Level	Provider	Channel	Provider
35148		1456	1456	2024-12-27 08:04:51	12	Info	Microsoft-Windows-Kernel-General	System	

Event Id - 12 And Provider - Microsoft-Windows-Kernel-General

Total lines 25,234 Visible lines 1 Open files 1 Search options

Timeline Explorer v2023.10

File Task Filter View Help

Timeline Explorer

Drag a column header here to group by that column

Enter text to search

Find

Line	Tag	Record Number	Event Record Id	Time Created	Event	Level	Provider	Channel	Provider
35139		1455	1455	2024-12-24 23:14:09	13	Info	Microsoft-Windows-Kernel-General	System	

Event Id - 13 And Provider - Microsoft-Windows-Kernel-General

Total lines 25,234 Visible lines 1 Open files 1 Search options

Since the EvtxECmd tool normalizes by default all timestamps in the UTC time zone when exporting the CSV, the exact times were pulled directly from the Time Created column. Therefore, the answers are as follows:

- **OS Shutdown-Event ID 13: 2024-12-24 23:14:09 UTC**

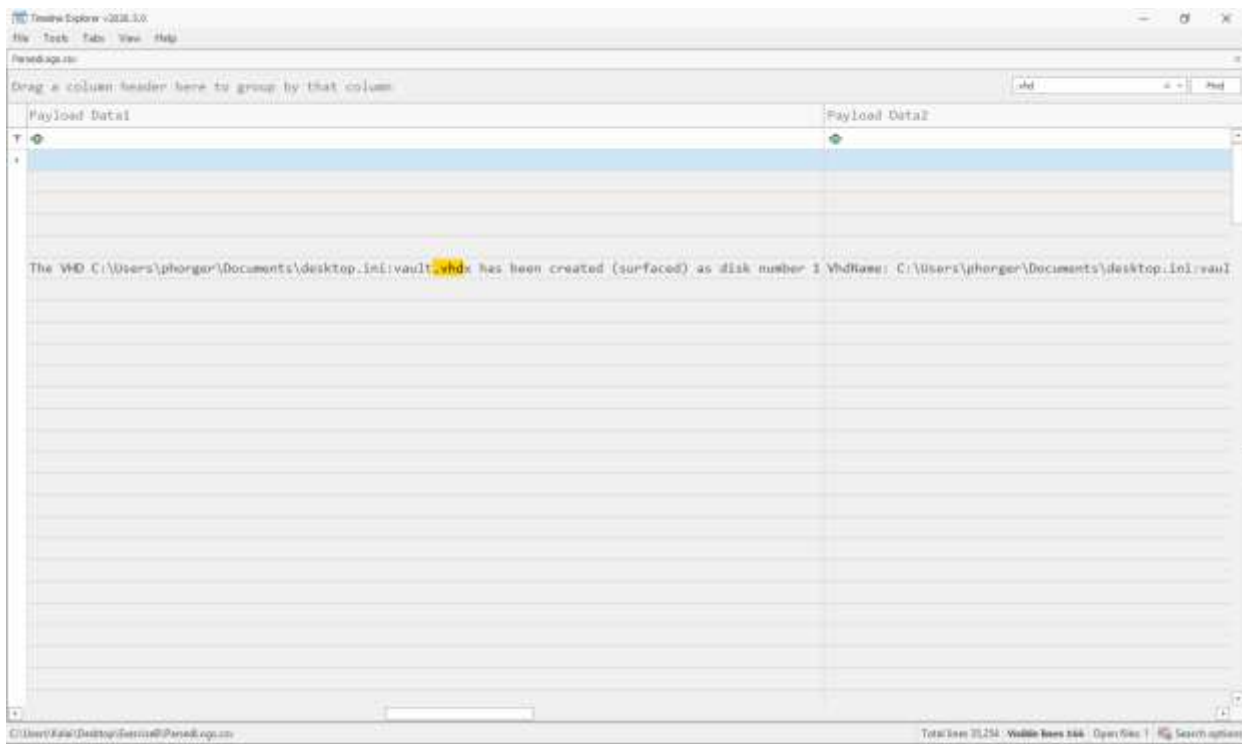
- **OS Start-Event ID 12: 2024-12-27 00:04:51 UTC**

It is worth noting that the chronological sequence is completely consistent, indicating that the system ended its operation on 24/12/2024 and restarted three days later, on 27/12/2024.

- 2) What is the route of the file in question, what is its name, as well as what is the date it was allegedly created?

Answer:

This question concerns the identification of a classic anti-forensics technique, in which one file is hidden inside another using the Alternative Data Streams (ADS) of the file system. Through this method, the hidden file is not visible while simply browsing the operating system folders. To locate the hidden virtual disk (VHD/VHDX), it was not necessary to specify a specific Event ID. Instead, the search function of Timeline Explorer was used. The search revealed the classic format of a file hidden via ADS. In the "Payload" field of the detected record, the use of the colon (:) "betrayed" that the vault.vhdx file had been attached and hidden behind the seemingly innocent desktop.ini file.



In order to determine the exact date and time when the record was created, the 'Time Created' column of the same selected row (row 31918) was examined. Since the export tool (EvtxECmd) converts all timestamps to UTC time zone by default when the CSV is produced, the displayed value is already in the correct format. The value shown in this column is 2024-12-24 22:57:32.

Line	Tag	Record Number	Event Record Id	Time Created	Event	Level	Provider	Channel	Pro
31925		232	232	2024-12-24 22:57:32	12	Info	Microsoft-Windows-VHDP	Microsoft-Windows-VHDP	
31924		231	231	2024-12-24 22:57:32	23	Info	Microsoft-Windows-VHDP	Microsoft-Windows-VHDP	
31923		230	230	2024-12-24 22:57:32	22	Info	Microsoft-Windows-VHDP	Microsoft-Windows-VHDP	
31921		228	228	2024-12-24 22:57:32	23	Info	Microsoft-Windows-VHDP	Microsoft-Windows-VHDP	
31920		227	227	2024-12-24 22:57:32	22	Info	Microsoft-Windows-VHDP	Microsoft-Windows-VHDP	
31918		225	225	2024-12-24 22:57:32	1	Info	Microsoft-Windows-VHDP	Microsoft-Windows-VHDP	
31917		224	224	2024-12-24 22:57:32	22	Info	Microsoft-Windows-VHDP	Microsoft-Windows-VHDP	
31916		223	223	2024-12-24 22:57:32	22	Info	Microsoft-Windows-VHDP	Microsoft-Windows-VHDP	
31915		222	222	2024-12-24 22:57:32	24	Info	Microsoft-Windows-VHDP	Microsoft-Windows-VHDP	
31914		221	221	2024-12-24 22:57:32	22	Info	Microsoft-Windows-VHDP	Microsoft-Windows-VHDP	
31913		220	220	2024-12-24 22:57:32	23	Info	Microsoft-Windows-VHDP	Microsoft-Windows-VHDP	
31912		219	219	2024-12-24 22:57:32	22	Info	Microsoft-Windows-VHDP	Microsoft-Windows-VHDP	
31911		218	218	2024-12-24 22:57:32	28	Info	Microsoft-Windows-VHDP	Microsoft-Windows-VHDP	
31910		217	217	2024-12-24 22:57:32	27	Info	Microsoft-Windows-VHDP	Microsoft-Windows-VHDP	
31909		216	216	2024-12-24 22:57:32	24	Info	Microsoft-Windows-VHDP	Microsoft-Windows-VHDP	
31908		215	215	2024-12-24 22:57:32	22	Info	Microsoft-Windows-VHDP	Microsoft-Windows-VHDP	
31907		214	214	2024-12-24 22:57:32	25	Info	Microsoft-Windows-VHDP	Microsoft-Windows-VHDP	
31906		213	213	2024-12-24 22:57:32	12	Info	Microsoft-Windows-VHDP	Microsoft-Windows-VHDP	
31905		212	212	2024-12-24 22:57:32	22	Info	Microsoft-Windows-VHDP	Microsoft-Windows-VHDP	
31904		211	211	2024-12-24 22:57:32	22	Info	Microsoft-Windows-VHDP	Microsoft-Windows-VHDP	
31903		210	210	2024-12-24 22:57:32	23	Info	Microsoft-Windows-VHDP	Microsoft-Windows-VHDP	
31902		209	209	2024-12-24 22:57:32	22	Info	Microsoft-Windows-VHDP	Microsoft-Windows-VHDP	
31900		207	207	2024-12-24 22:57:32	23	Info	Microsoft-Windows-VHDP	Microsoft-Windows-VHDP	
31899		206	206	2024-12-24 22:57:32	22	Info	Microsoft-Windows-VHDP	Microsoft-Windows-VHDP	
31895		202	202	2024-12-24 22:57:32	20	Info	Microsoft-Windows-VHDP	Microsoft-Windows-VHDP	
31894		201	201	2024-12-24 22:57:32	27	Info	Microsoft-Windows-VHDP	Microsoft-Windows-VHDP	

Based on the above digital findings, the answer to question 2 is as follows:

- **File path:** C:\Users\phorger\Documents\desktop.ini
- **Hidden File Name (ADS):** desktop.ini:vault.vhdx
- **Creation date and time (UTC):** 2024-12-24 22:57:32

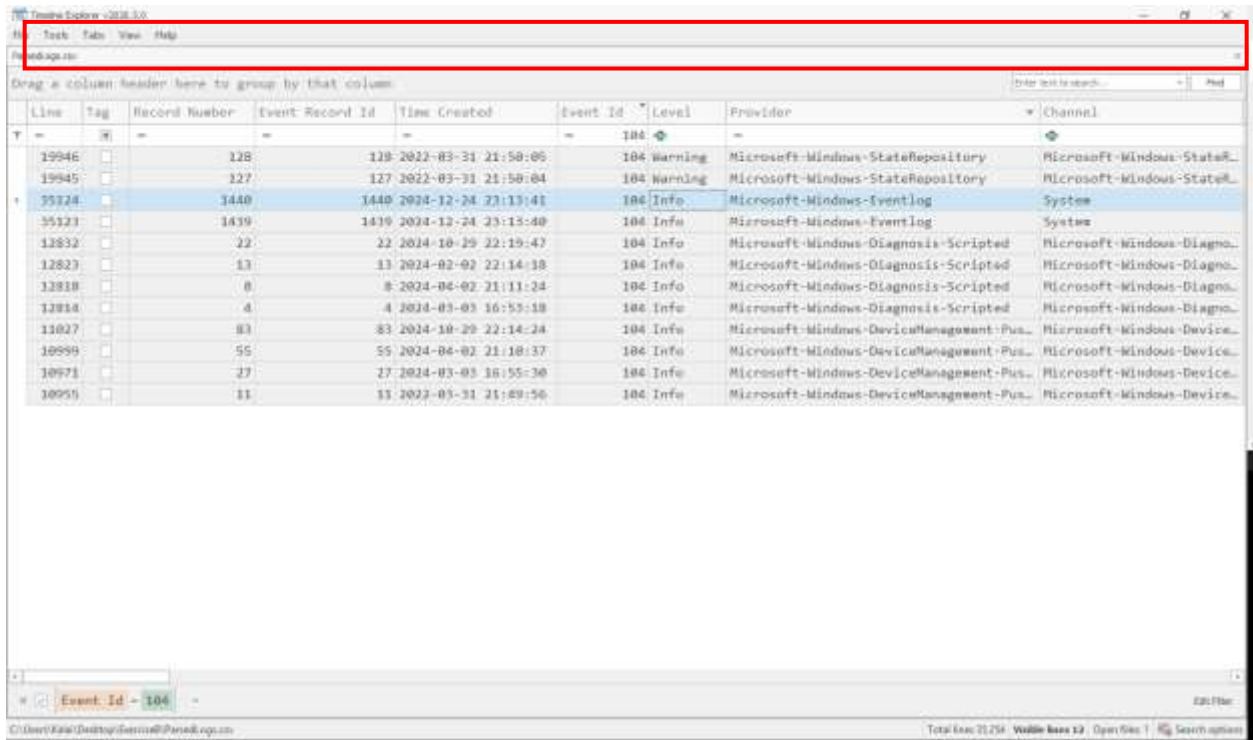
3) Identify which logs the user in question cleared the records from, and when those actions took place?

Answer:

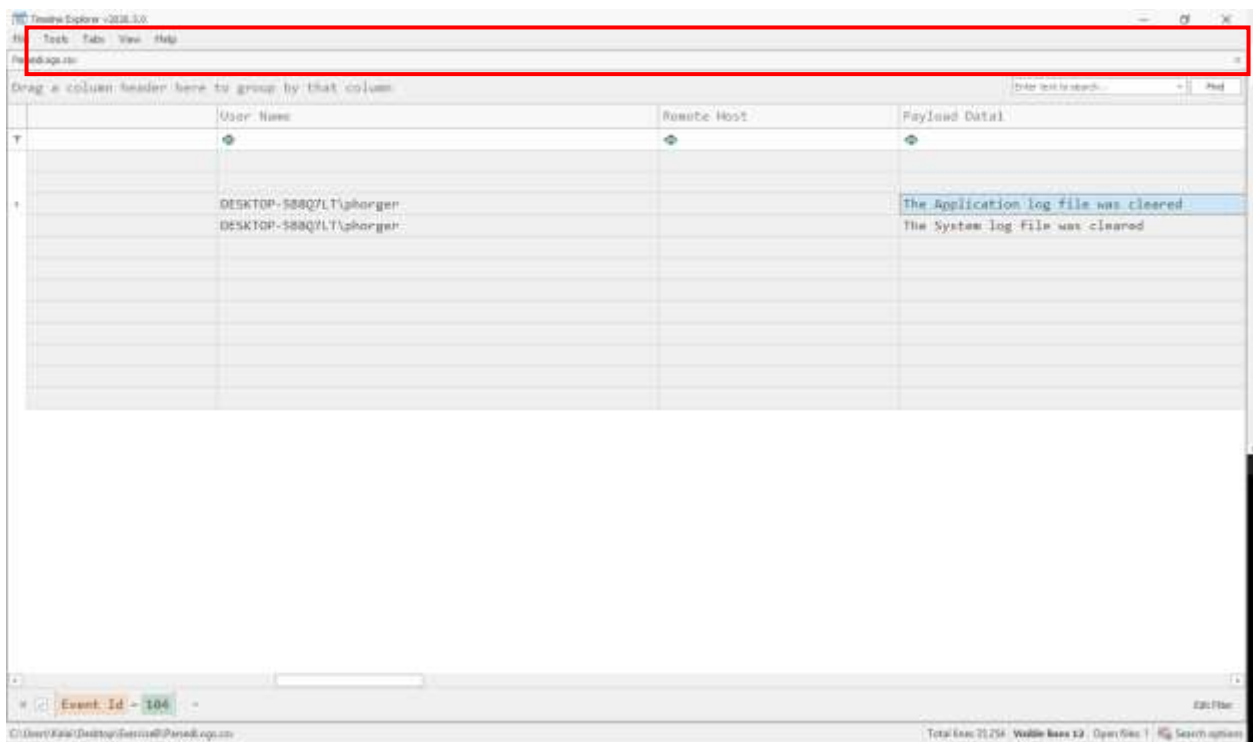
This question addresses one of the most critical indications of a system breach, the deliberate purge of logs. This action is a common technique for masking traces. In Windows operating systems, the very action of clearing logs generates specific event IDs by default for auditing purposes. Event ID 104 is used to clean up the System and Application files, and Event ID 1102 is used to clean up the Security file.

Initially, the search in the Timeline Explorer environment focused on Event ID 104. Filtering the results, two critical records were identified by the "Microsoft-Windows-Eventlog"

Provider. As confirmed by the analysis of the Payload field, the Application log was cleared on 2024-12-24 23:13:40 UTC. Exactly one second later, on 2024-12-24 23:13:41 UTC, the system log was cleared.



Line	Tag	Record Number	Event Record Id	Time Created	Event Id	Level	Provider	Channel
19946		128	128	2022-03-31 21:58:05	104	Warning	Microsoft-Windows-StateRepository	Microsoft-Windows-StateR...
19945		127	127	2022-03-31 21:58:04	104	Warning	Microsoft-Windows-StateRepository	Microsoft-Windows-StateR...
55324		1440	1440	2024-12-24 23:13:41	104	Info	Microsoft-Windows-Eventlog	System
55323		1439	1439	2024-12-24 23:13:40	104	Info	Microsoft-Windows-Eventlog	System
12832		22	22	2024-10-25 22:15:47	104	Info	Microsoft-Windows-Diagnosis-Scripted	Microsoft-Windows-Diagno...
12823		13	13	2024-02-02 22:14:18	104	Info	Microsoft-Windows-Diagnosis-Scripted	Microsoft-Windows-Diagno...
12818		8	8	2024-04-02 21:11:24	104	Info	Microsoft-Windows-Diagnosis-Scripted	Microsoft-Windows-Diagno...
12814		4	4	2024-03-03 16:55:18	104	Info	Microsoft-Windows-Diagnosis-Scripted	Microsoft-Windows-Diagno...
11027		83	83	2024-10-29 22:14:24	104	Info	Microsoft-Windows-DeviceManagement-Pus...	Microsoft-Windows-Device...
10999		55	55	2024-04-02 21:18:37	104	Info	Microsoft-Windows-DeviceManagement-Pus...	Microsoft-Windows-Device...
10971		27	27	2024-03-03 16:55:30	104	Info	Microsoft-Windows-DeviceManagement-Pus...	Microsoft-Windows-Device...
10955		11	11	2022-03-31 21:09:50	104	Info	Microsoft-Windows-DeviceManagement-Pus...	Microsoft-Windows-Device...



User Name	Remote Host	Payload Data
DESKTOP-588Q7LT\phorger		The Application log file was cleared
DESKTOP-588Q7LT\phorger		The System log File was cleared

Since the Security log is the primary target when masking traces (as it contains authentication data), the investigation was extended to Event ID 1102. Because the numbering is not unique (e.g. irrelevant records from ShellCommon are displayed), the results were further filtered by 'Provider' (Microsoft-Windows-Eventlog) and 'Channel' (Security). This combination successfully revealed one record, which confirms the clearing of the Security log on 2024-12-24 23:13:44 UTC. Additionally, this particular registration offers crucial elements of accountability. In the 'Map Description' column, the action is explicitly described as 'Event log cleared', while the 'User Name' column indicates that the deletion was performed from the phorger user's account (on the DESKTOP-5B8Q7LT system).

- **Application log: 2024-12-24 23:13:40**
- **System log: 2024-12-24 23:13:41**
- **Security log: 2024-12-24 23:13:44**

4) When was the connection with administrator privileges made for the user "phorger", according to the logs?

Answer:

To determine the exact time when the phorger user (who was previously detected clearing logs) logged into the system with elevated Administrator privileges, the analysis focused on authentication events.

On Windows systems, a typical, successful login is recorded with Event ID 4624 (Successful Logon). However, when special privileges or administrator privileges are assigned to this connection, the operating system simultaneously creates a supplemental record with Event ID 4672 (Special privileges assigned to new logon). This very identifier (Event ID 4672) was the key to the investigation.

The methodology in Timeline Explorer included combo filtering. Specifically, filter 4672 was set in the Event Id column in order to isolate only connections with special permissions. At the same time, because the system also performs its own processes as an administrator, an additional filter was applied to the User Name column with the value phorger, in order to isolate the specific suspicious account. This targeted search returned with absolute accuracy one (1) unique login record that met the criteria. By checking the "Time Created" column of this record, the timestamp of the action was extracted.

Timeline Explorer v2023.0.0

File Task Fats View Help

ParadeLog.ru

Drag a column header here to group by that column

Enter text to search: [] Find

Line	Tag	Record Number	Event Record Id	Time Created	Event Id	Level	Provider	Channel	Process Id	Computer
34889		8113		8113 2024-12-27 00:06:49	4672		Microsoft-Windows-Security-Auditing	Security		664-DESKT

Event Id = 4672 and User Name = DESKTOP-588071.T\phorger (5-1-5-21-1540112901-4150280657-3740420248-1000)

Ch Desktop\Desktop\Timeline\ParadeLog.ru

Total lines 21,234 Visible lines 1 Open files 1 Search options

Therefore, based on the Security log, **Login with admin privileges for phorger took place on 2024-12-27 00:06:49 UTC.**

- 5) Based on the logs, what was the operating duration (in seconds) of the system on "2024-04-05"?

Answer:

In order to accurately determine the duration of system operation on this date, it is not necessary to manually and unsuccessfully subtract the start time from the shutdown time. On Windows systems, the operating system itself takes care of automatically recording this metric (to the second) through a special event ID, Event ID 6013.

To retrieve this information in the Timeline Explorer environment, a new filter was applied to the "Event Id" column with a value of 6013. In order to isolate the requested time period set by the voiceover, an additional search filter was applied to the "Time Created" column for the date 2024-04-05. This methodology immediately highlighted the desired registration (line 35221).

Timeline Explorer v2023.3.0

File Task Filter View Help

Parse logs as:

Drag a column header here to group by that column:

Enter text to search: [] Find

Line	Tag	Record Number	Event Record Id	Time Created	Event Id	Level	Provider	Channel	Process Id	Computer
35221		1537	1537	2024-04-05 01:27:05	6013	Info	EventLog	System		DESK
35118		1454	1454	2024-12-27 08:06:14	6013	Info	EventLog	System		DESK

Event Id = 6013

Total lines: 35,234 | Visible lines: 2 | Open files: 1 | Search options

By examining the data of this record in the column "Payload Data1", the exact information generated by the system was retrieved. The text explicitly records "The system uptime is 1078 seconds".

Timeline Explorer v2023.3.0

File Task Filter View Help

Parse logs as:

Drag a column header here to group by that column:

Enter text to search: [] Find

	Payload Data1	Process Id
	The system uptime is 1078 seconds	System
	The system uptime is 84 seconds	System

Event Id = 6013

Total lines: 35,234 | Visible lines: 2 | Open files: 1 | Search options

Therefore, based on the official Event Logs, the definitive answer is that **The operating time of the system on 2024-04-05 was 1078 seconds.**

Epilogue

Analysis of the logs revealed a comprehensive and highly suspicious sequence of events in the system. Through our research with the EvtxECmd and Timeline Explorer tools, the puzzle of actions was successfully synthesized, starting with the detection of anti-forensics techniques, such as the discovery of the vault.vhdx virtual disk, which was hidden behind the seemingly innocent desktop.ini file through the use of Alternative Data Streams (ADS). This action, which is a classic method for unauthorized data extraction, was accompanied by the masking of traces by the phorger user, who was identified as bulk deleting the evidence, clearing the Application, System (Event ID 104) and Security (Event ID 1102) logs within a time window of just four seconds. Finally, the complete mapping of the operation history and access rights allowed the accurate determination of the start, shutdown and duration of the system, while highlighting the moment when the suspicious user acquired administrator privileges (Event ID 4672), which allowed him to complete the destruction of the logs.